

=====

SECURE OTA FIRMWARE UPDATE FOR ESP32 - PROJECT GUIDE

Simple explanation of every important file and the full working flow

=====

PART 1 - WHAT IS THIS PROJECT?

We update the firmware of an ESP32 board using WiFi. No USB cable is needed.

But sending firmware over WiFi is dangerous. Anybody on the network can change the file in the middle. So we protect the firmware in three ways:

1. LOCK IT - We encrypt the firmware. Nobody can read it.
 We use Ascon-AEAD128.
2. SEAL IT - We take a fingerprint of the firmware. If even one byte
 changes, the fingerprint changes.
 We use Ascon-Hash256.
3. SIGN IT - We put our signature on it. Only we can make this
 signature. Anybody can check it, but nobody can copy it.
 We use Ed25519.

The ESP32 checks all three. If any one check fails, it throws the update away and keeps running the old firmware. Nothing is broken.

Ascon is not our own invention. It is the official lightweight cryptography standard of NIST (SP 800-232). It is made for small chips like ESP32.

PART 2 - THE THREE PARTIES

YOUR PC (build machine)

Has the secret signing key. Builds the firmware, locks it, signs it.
This is the boss. Only this machine can approve firmware.

THE SERVER (server/app.py)

Has NO keys at all. It only stores files and gives them when asked.
Even if a hacker takes full control of the server, he cannot make
fake firmware, because he does not have the signing key.

THE ESP32 BOARD

Has the public key inside its own program. It uses this to check the
signature. It can check, but it cannot sign. So a stolen board is
not a problem.

This separation is the main strength of the project.

PART 3 - THE FULL FLOW (STEP BY STEP)

STEP 1 - BUILD

You open the website and type a version number, like 12.0.0.
You press "Build firmware".
The server writes this number inside main/app_config.h and runs the ESP-IDF compiler. After 1 minute you get a file: build/secure_ota.bin
This is plain firmware. Not yet safe. Anybody can read it.

STEP 2 - MAKE IT SECURE

You press "Create secure package".
Now three things happen on your PC:

- a) Ascon-Hash256 takes the fingerprint of the plain firmware.
- b) Ascon-AEAD128 locks (encrypts) the firmware with a secret key and a fresh random number called a nonce.
- c) Ed25519 puts your signature on the header.

The result is one file: firmware_v12.0.0.sota
This file is safe to send anywhere.

STEP 3 - PUBLISH

You press "Publish". The file is copied to server/packages/.
Now the server will give it to any board that asks.
Before publishing, the file is kept in server/staging/ where the board cannot see it. So you can prepare quietly and release when ready.

STEP 4 - THE BOARD TAKES IT

Every 60 seconds the ESP32 asks the server: "Is there a new version?"
If yes, it downloads and checks in this exact order:

- CHECK 1 Signature (Ed25519)
Is this really from my owner? If no -> REJECT and stop.
This happens BEFORE downloading the firmware. Very important.
- CHECK 2 Version (anti-rollback)
Is this newer than what I have? If it is older -> REJECT.
This stops a hacker from pushing an old firmware that had a bug in it.
- CHECK 3 Decrypt and tag (Ascon-AEAD128)
Can I unlock it, and is the tag correct?
If no -> REJECT. Somebody changed the file.
- CHECK 4 Fingerprint (Ascon-Hash256)
Does the unlocked firmware match the fingerprint that was signed? If no -> REJECT.

Only after ALL FOUR pass, the board writes the firmware into a spare partition and reboots into it.

IMPORTANT: The old firmware is never touched during all this. If anything fails, the board simply continues with the old firmware. No brick.

STEP 5 - SAFETY AFTER REBOOT

After reboot the new firmware is on "probation". It must connect to WiFi and prove it is working. Only then it is marked as good. If it fails, the board automatically goes back to the old firmware on the next restart.

PART 4 - ONE LINE PICTURE

```
secure_ota.bin (plain firmware)
  |
  | Ascon-Hash256 -> fingerprint
  | Ascon-AEAD128 -> locked firmware + tag
  | Ed25519       -> signature
  v
firmware_v12.0.0.sota
  |
  | HTTP over WiFi
  v
ESP32 board
  |
  | signature? version? decrypt? fingerprint?
  | any one fails -> throw away, keep old firmware
  v
write spare partition -> reboot -> new firmware running
```

===== A. FIRMWARE - THE CODE RUNNING ON THE ESP32 =====

main/main.c

The starting point of the board. Like main() in C.
It starts everything in correct order: storage, keys, WiFi, then the OTA task. It also runs the blinking LED and the system monitor that prints uptime and free heap on the serial console.
The LED blink speed tells you the version. Version 12 blinks faster than version 10. So you can see the update happened, without reading any log.

main/app_config.h

The settings file. THE MOST IMPORTANT NUMBERS ARE HERE:
FIRMWARE_VERSION_MAJOR / MINOR / PATCH -> the version, like 12.0.0
SECURITY_VERSION -> anti-rollback counter
Also has timings: how often to check for update, download chunk size, WiFi timeout, monitor print interval.

main/device_config.h

WiFi name, WiFi password, and the server address.
This file is secret, so it is NOT uploaded to git.
Copy device_config.h.example and fill your own values.

main/crypto_config.h

The public key of the owner and the shared Ascon key, put inside the firmware at build time. Also secret, also not in git.
This is the "root of trust" of the board.

main/ota_manager.c / .h

THE BRAIN OF THE UPDATE. This is the most important firmware file.
It runs the state machine:
IDLE -> CHECK -> METADATA -> SIGNATURE_VERIFY -> VERSION_VERIFY
-> DOWNLOAD -> DECRYPT -> HASH_VERIFY -> INSTALL -> REBOOT
It downloads in 4 KB pieces, decrypts each piece, and writes it straight to flash. So the full firmware is never kept in RAM.
Total extra RAM needed is only about 9 KB, even for a 900 KB firmware.

main/wifi_manager.c / .h

Connects to WiFi. Retries if it fails. Tells other parts when the network is ready.

main/device_keys.c / .h

Loads the Ascon key from NVS storage. On the very first boot, it takes the key from crypto_config.h and saves it into NVS.
It NEVER prints the key. It only prints a short fingerprint of the key, so you can confirm it is correct without exposing it.

main/version_manager.c / .h

Remembers the highest version the board has accepted. This is stored in NVS so it survives reboot. This is what stops rollback attacks.

main/device_report.c / .h

Sends a heartbeat to the website every 5 seconds (1.5 seconds during an update) so the dashboard can show live status.
It can receive only THREE commands: CHECK_UPDATE, START_OTA, REBOOT.
Nothing else. The server cannot tell the board to run any other code.
This keeps the server untrusted, which is the design.

===== B. CRYPTO COMPONENTS - THE MATH =====

components/ascon/

Ascon-Hash256 (fingerprint) and Ascon-AEAD128 (lock/unlock).
Written in plain C. Only 1,982 bytes of flash for both.
No ESP-IDF code inside, so the same file can be tested on the PC.
Main files: src/ascon_hash256.c, src/ascon_aead128.c, src/permutations.c

components/ed25519/

Checks the signature. Taken from TweetNaCl, a famous trusted library.
We did not write our own crypto. That would be a mistake.

Main files: src/ed25519_verify.c, src/tweetnacl.c

components/ota_package/

Reads the .sota file, understands its header, and runs the four checks in the correct order. If anything is wrong it stops immediately and makes sure no secret data is left in memory.

Main file: src/ota_package.c

===== C. PC SIDE - MAKING THE PACKAGE =====

sotalib/ascon.py

Same Ascon algorithm, but in Python. Used to make the package on PC.

sotalib/package.py

Defines the .sota file format. Builds it and reads it.

Both the PC and the ESP32 follow this same format exactly.

tools/generate_keys.py

Makes the Ed25519 key pair and the Ascon key. RUN THIS FIRST, ONCE.

Use --write-device-config so the board also gets the public key.

tools/create_ota_package.py

Takes a plain .bin and makes the safe .sota package.

This is the tool that does the real hash + encrypt + sign work.

tools/verify_package.py

Checks a .sota package on the PC, exactly the same way the board does.

Prints each check as PASS or FAIL. Very useful for demo.

tools/tamper_package.py

Creates ATTACK packages on purpose. 9 different attacks:

flip-ciphertext, flip-metadata, bad-signature, foreign-signer, wrong-key, hash-mismatch, rollback, truncate, bad-magic.

Used to prove the system rejects bad firmware.

tools/simulate_device.py

A fake device, for testing the website without hardware.

It does no real crypto. Only for checking the UI.

===== D. SERVER AND WEBSITE =====

server/app.py

The OTA server. Gives the package to the board.

IT HOLDS NO KEYS. It only reads the public part of the file header.

Endpoints used by the board:

/api/firmware/latest	-> which version is available
/api/firmware/<version>/package	-> download the package
/health	-> is server alive

server/dashboard/__init__.py

All the website routes (the web addresses). Connects the page to everything else - build, upload, package, publish, device heartbeat, attack tests, history.

server/dashboard/builder.py

Runs "idf.py build" from the website. Writes the version numbers into

app_config.h first, so the source always matches the built firmware.

The build runs in the background so the browser does not hang.

server/dashboard/uploads.py

SECURITY FOR UPLOADED FILES. Very important file.

What it does with every uploaded .bin:

- Cleans the file name. Removes folders, C:\, "..", spaces, special characters. So the file can never escape the upload folder.
- Limits size to 16 MB (largest ESP32 flash). Empty file rejected.
- Writes to a temporary file first, then moves. So a half-uploaded file never appears.
- Takes SHA-256 of the file. Same file uploaded twice is not stored twice.
- Reads the ESP32 header and reports what the file really is: project name, version, chip, compile date.

- If the file is source code (.ino), it says so clearly.
The upload folder is NEVER served on the web. You cannot download from it.

server/dashboard/packaging.py

Calls tools/create_ota_package.py in the background.
The signing key never comes to the browser. It stays in keys/.

server/dashboard/inventory.py

Lists the packages in server/packages/ (published) and server/staging/ (not yet published).

server/dashboard/sectest.py

The ATTACK TEST LAB. Runs tamper_package.py and verify_package.py to prove each attack is caught. Nothing hostile is ever published or sent to a board.

server/dashboard/db.py

SQLite database (server/ota.db). Keeps history only:
device heartbeats, uploaded files, packages, update results, security events. NO SECRET IS STORED HERE.

server/dashboard/logbus.py

Keeps the recent log lines in memory so the website can show them.

server/dashboard/templates/index.html

THE WEBSITE PAGE. One single page with everything:
- What is happening now (live device status)
- Step 1 Build, Step 2 Upload, Step 3 Package, Step 4 Publish
- How encryption and decryption work
- Attack tests
- How to use

server/dashboard/static/css/site.css -> the look of the page
server/dashboard/static/js/site.js -> the buttons and live updates

===== E. OTHER IMPORTANT FILES =====

partitions.csv

The flash memory map of the ESP32:
nvs - small storage for keys and version
otadata - which partition to boot
factory - firmware flashed by USB
ota_0 - spare slot 1
ota_1 - spare slot 2

Two spare slots is why the update is safe. New firmware goes to the free slot. Old firmware stays untouched until the new one works.
NOTE: 4 MB flash is compulsory. 2 MB will not fit.

keys/

ed25519_private.pem - SECRET. Only on your PC. Never share.
ed25519_public.pem - Goes inside the firmware. Safe to share.
ota_enc_key.hex - SECRET. Shared between PC and board.

tests/

test_ascon_kat.py - checks Ascon against 2114 official test vectors
test_package.py - checks the file format byte by byte
test_signature.py - checks the signature cannot be faked
test_negative.py - the 7 attack scenarios
test_server.py - server behaviour, and that it holds no keys
test_dashboard.py - website, upload security, build
Run all: python -m pytest tests/ -q (182 tests)

docs/

ARCHITECTURE.md - full design
PACKAGE_FORMAT.md - the .sota file format, byte by byte
SECURITY.md - what is protected and what is not (honest list)
DEMO.md - step by step demo script
WEB_INTERFACE.md - the website and upload security

FIRST TIME ONLY

```
pip install -r server/requirements.txt
python tools/generate_keys.py --write-device-config
copy main/device_config.h.example main/device_config.h
    (put your WiFi name, WiFi password, and your PC IP address)
idf.py set-target esp32
idf.py -p COM11 flash monitor
```

EVERY TIME

```
python server/app.py
Open http://localhost:8000/
Step 1 -> type next version -> Build firmware
Step 3 -> Create secure package
Step 4 -> Publish
Wait about 1 minute. Board updates by itself. LED blinks faster.
```

1. NEVER UPLOAD AN ARDUINO .bin

An Arduino sketch has only `setup()` and `loop()`. It has no WiFi manager and no OTA client. If you send it to the board, all four security checks will PASS (because you signed it correctly), the board will install it and boot it - and after that the board can never receive any update again. You will need a USB cable to recover.

If you want to add new code, put it inside `main/main.c` as a FreeRTOS task, then press Build on the website. Then your new code AND the OTA client both run together.

2. VERSION MUST ALWAYS INCREASE

The board installs only a NEWER version. Same or older is rejected. Security version can never go down. This is the anti-rollback rule.

3. THE VERSION YOU TYPE MUST MATCH THE VERSION YOU BUILT

If you build 12.0.0 but type 13.0.0 while packaging, the board will install it and then report 12.0.0. The dashboard will show "version mismatch". Using the Build step avoids this automatically.

4. A .bin IS NOT A .ino

.ino is source code that a human writes.

.bin is the compiled output, about 900 KB, first byte is 0xE9.

You cannot upload source code. It must be compiled first.

5. PLAIN HTTP IS ONLY FOR DEMO

For real use, run with `--https`. But note: even on plain HTTP, the signature, the tag and the fingerprint are still fully checked. A hacker on the network still cannot push fake firmware. HTTP only fails to hide that an update is happening.

6. DO NOT OPEN PORT 8000 TO THE INTERNET

There is no login on the website. Anybody on the same network can upload and publish. This is fine for a college lab network only.

PROTECTED AGAINST

- Somebody changing the firmware in the middle
- Somebody making fake firmware without our key
- Somebody sending an old firmware with a known bug (rollback)
- Wrong encryption key
- Broken or half-downloaded file
- A fully hacked server

NOT PROTECTED AGAINST

- Somebody who physically opens the chip and reads the flash. The Ascon key can be taken out. Then he can READ the firmware. But he still CANNOT make fake firmware, because the signing key is not on the board. For a real product we need eFuse + Flash Encryption.
- A board that is already fully hacked
- If your PC with the private key is stolen
- Denial of service - somebody can simply block the network

This is an academic project. The cryptography is standard and correct and is tested against official test vectors. But it is not audited. We do not claim it is unbreakable.

END OF GUIDE
